Construcción de Compiladores
Septiembre, 2026

Proyecto 2
Generación de Código Intermedio para Compiscript

Descripción

Esta actividad corresponde a la fase de **generación de código intermedio** (CI) en la construcción de un compilador para **Compiscript**, un lenguaje basado en un subconjunto de TypeScript. A partir de las estructuras de datos producidas por el análisis semántico (árbol sintáctico y tabla de símbolos), se debe generar una representación intermedia del programa fuente en forma de **código de tres direcciones** (TAC). Esta representación servirá como base para la posterior generación de código assembler u objeto.

- **Generación de Código Intermedio:** Acciones semánticas sobre el árbol sintáctico que traducen cada construcción de Compiscript a instrucciones de tres direcciones.
- **Manejo de Temporales:** Algoritmo de asignación y reciclaje de variables temporales durante la traducción de expresiones.
- **Tabla de Símbolos Extendida:** Información adicional necesaria para la generación de código (direcciones, desplazamientos, tamaños, etiquetas) y soporte para entornos de ejecución mediante registros de activación.

Especificaciones

El generador de código intermedio debe cumplir, como mínimo, con lo siguiente:

- **Diseño del Código Intermedio:**
  - Definición de un conjunto de instrucciones de tres direcciones (asignación, operaciones binarias y unarias, copia, saltos condicionales e incondicionales, etiquetas, llamadas a procedimientos, acceso indexado, etc.).
  - La sintaxis es a discreción del diseñador, pero debe ser consistente y estar documentada.
- **Expresiones y Asignaciones:**
  - Traducción de expresiones aritméticas (`+`, `-`, `*`, `/`) respetando precedencia y asociatividad.
  - Traducción de expresiones lógicas (`&&`, `||`, `!`) y de comparación (`==`, `!=`, `<`, `<=`, `>`, `>=`).
  - Traducción de declaraciones y asignaciones de variables y constantes.
- **Control de Flujo:**

  - Traducción de `if/else`, `while`, `do-while`, `for` y `switch` mediante etiquetas y saltos.
  - Traducción de `break` y `continue` hacia las etiquetas correspondientes del bucle que los contiene.

- **Funciones y Procedimientos:**
  - Traducción de la definición de funciones, paso de parámetros, llamadas y valores de retorno.
  - Soporte para funciones recursivas.
  - Soporte para funciones anidadas y closures, considerando el acceso a variables del entorno de definición.

- **Clases y Objetos:**
  - Traducción de la creación de objetos e invocación del constructor.
  - Traducción del acceso a atributos y de la invocación de métodos, incluyendo el uso de `this`.

- **Listas y Estructuras de Datos:**
  - Traducción de la creación de listas y del acceso a sus elementos mediante índices.

- **Variables Temporales:**
  - Algoritmo de asignación de temporales durante la traducción de expresiones.
  - Reciclaje de temporales una vez que su valor ha sido consumido, minimizando la cantidad de temporales utilizados.

- **Tabla de Símbolos y Entornos de Ejecución:**
  - Complementar cada símbolo con la información necesaria para la generación de código (direcciones de memoria o desplazamientos, tamaños, etiquetas).
  - Soportar los ambientes y entornos en tiempo de ejecución mediante registros de activación (parámetros, variables locales, temporales, valor y dirección de retorno, enlaces de control y de acceso).
  - La tabla de símbolos debe interactuar con cada fase de la compilación.

Entrada:

- Archivo fuente de Compiscript con extensión `.cps`.

Salida:

- Código intermedio (TAC) generado a partir del programa fuente.
- Reporte de errores encontrados durante la compilación.
- Estado de la tabla de símbolos con la información agregada para la generación de código (direcciones, desplazamientos, registros de activación).

Construcción de Compiladores
Septiembre, 2026

Funcionamiento del Programa

1. El usuario carga o escribe un archivo `.cps` a través del IDE.
2. El analizador léxico y sintáctico (ANTLR) construye el árbol sintáctico.
3. El analizador semántico recorre el árbol validando las reglas del lenguaje y construyendo la tabla de símbolos.
4. Si no existen errores, el generador recorre el árbol aplicando las acciones semánticas de traducción, consultando y complementando la tabla de símbolos, y produce el código intermedio.
5. El programa muestra el código intermedio generado o, en su caso, los errores encontrados con su tipo y ubicación.
6. El usuario puede ejecutar la batería de pruebas para validar casos exitosos y fallidos.

Entregables

- Un repositorio de GitHub, con commits individuales que evidencien claramente la contribución de cada integrante (no se permite compartir commits en conjunto).
- Batería de pruebas que valide casos exitosos y fallidos de la generación de código intermedio, presente y funcional al momento de la evaluación.
- Documentación de la arquitectura de la implementación y documentación de cómo ejecutar el compilador.
- Documentación detallada del lenguaje intermedio diseñado, con ejemplos de traducción y los supuestos considerados. Esta documentación servirá a los calificadores para comprender las decisiones de diseño y verificar la implementación.
- IDE funcional que permita escribir y compilar código Compiscript.

Construcción de Compiladores
Septiembre, 2026

Evaluación

Requisitos para Calificación

Para poder optar a calificación, el programa debe funcionar correctamente el día de la presentación. El proyecto debe estar correctamente documentado y organizado.

Componente	**Descripción**	**Puntos**
Diseño de CI	Definición y documentación del lenguaje intermedio, con ejemplos y supuestos de traducción	25 pts
Generación de TAC	Traducción de Compiscript a código de tres direcciones, manejo de temporales y batería de pruebas	65 pts
Tabla de Símbolos	Nuevas adiciones para generación de código y soporte de registros de activación	10 pts
	Total	**100 pts**